LLM Inference (Ref:- lentaml.com/llm)

2 phases → PREFILL (Filling up KV cache) — Compute Bound
→ DECODE (R/W to KV cache) — Memory Bound.

Macro (Distributed Inference) — Regional routing, Scheduling, Scaling
Micro (Distributed Inference) — KV cache, Spec. Decode, C.B.

---

* Choosing the model → Dense (All parameters are active)
→ MoE (Only some depending on the i/p) — less in memory — So scalable.

* Choosing a GPU :- Memory (GB) = $P * \left(\frac{\theta}{8}\right) * (1 + \text{Overhead})$
↓
KV cache, activation ~30%

* Finetuning :- Use Unsloth with LoRA (Add small trainable matrices to fine-tune LLMs instead of changing all params)
(AWQ, GPTQ)

* Quantization :- FP32 → FP16 → INT8 (Only on weights)

Pruning :- Remove parameters that are less important
↓
(weights, attention layers, neurons)

* Need for multiple inference runtimes?
Different needs expect different " ".
- High throughput, batching :- vLLM, SGLang, TGI, MAX
- Mobile deployment :- llama.cpp
- Local experiment :- " " or Ollama.

Inference Optimization

* Key metrics for LLM inference:-

- **Latency** (Time taken for model to respond to a request)
  - Time to First Token (TTFT): TT to generate 1st token
  - Total Latency (E2EL): Time from sending the request to receiving the ~~response~~ final token on user end.

~~E2EL =~~

  - Token Generation time = E2EL − TTFT
    (TT to stream all tokens except the 1st token)
  - ~~Inter~~ Time per Output Token = Avg. time taken b/w generating each subsequent token (excluding TTFT).

$$TPOT = \frac{E2EL - TTFT}{\text{Total o/p tokens} - 1}$$

  - Inter token latency (ITL): Exact pause b/w 2 tokens

$$\text{Avg } ITL = TPOT$$

- **Throughput** (How much work an LLM can do in a given period)

- Requests per Second (RPS)

$$RPS = \frac{\text{Total completed requests}}{T_1 - T_2}$$

($T_1 - T_2$: Time window in sec.)

Impacted by:
- Prompt length
- Model size / hardware
- Optimizations
- Latency per request

- Tokens Per Second (TPS)
  - Input TPS (Input tokens processed)
  - Output TPS (O/p " generated by the model/sec)

Impacted by
- Batch size
- KV cache
- Seq/Prompt length
- GPU mem BW & C.U

- Goodput (Refines the idea of throughput)

It measures how many requests per sec the LLM completes while meeting service-level objective (SLO)

↓

95% of chat should've TTFT < 200ms?

---

* LLM Performance Benchmarks

When to run these?

- Comparing different models (DeepSeek V1 Vs. GPT-OSS)
- Evaluating inference frameworks (vLLM v. SGLang)
- Testing infrastructure changes (A10G → H100)
  (Onpremis Vs. cloud)
- Measuring Optimization gains (KV cache, S.D)

---

* Batching

Instead of responding to 1 request at a time, which is inefficient, batching helps us with bundling 'n' batch of requests and process/generate them at a time.

Batch
- "Hi, there?" — User 1
- "Explain Gravity" — User 2
- "Time in IST" — User 3

Types:
- Static Batching (Batch-wise)
- Dynamic " (Batch + Time out)
- Continuous " (Token-wise scheduling)

* Flash Attention :-

The standard attn is fundamentally a memory-bound problem.

$$Attn(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_K}}\right)V$$

If N = seq. length, the $QK^T = N \times N$

If N = 4096, then its 16 million elements we have to read & write to HBM.

Flash Attn fuses the matmul + Softmax + matmul, by using blocked/tiled calculation and online softmax.
(Kernel fusion)

---

* Paged Attention :-

Memory efficient implementation of attn. (KV cache).
KV cache needs to be stored & retrieved for every token a model generates. Instead of allocating KV cache in a big contiguous block (where it might not need all), we can allocate it in non-contiguous locations using Paging (& a look up table). [Saves HBM in GPU]

* Speculative Decoding

- It speeds us inference by having 2 models:
  - Draft Model: Smaller, fast model that drafts tokens ahead
  - Target Model: Larger model that verifies in parallel.

- Why Speculative Decoding?

LLMs are auto-regressive (one token at a time), so its sequential.
Some of the disadv of seq. process:-

- High Inter Token Latency (ITR)
- Low GPU Utilization (Idel GPUs).

We could parallelize some of the generation process.

→ LLMs are Memory Bound
→ Some tokens are easier to predict than others (From based on context)

- How it works:

At a high-level it runs in a loop:

1. Draft model predicts the next K-tokens
2. Target model verifies these K-tokens in parallel
3. Target " accepts the longest prefix of these K-tokens.
4. If it accepts 'h' tokens, then it generates $(h+1)^{th}$ token (so the generation remains on track)
5. The process repeats: Draft model predicts next K-tokens based on the extended token.

I/P seq

The weather is → quite nice today (Draft)
The weather is → ~~quite~~ ~~nice~~ × (Target)
The weather is → quite nice outside (Target)
The weather is → quite nice outside for a walk (Draft)

- Understanding the performance of SD

Key SD can accelerate LLM inference, but only when the draft and target model align. So verify there in vLLM before.

- Key Metrics

1. Acceptance Rate (α): Probability of draft tokens accepted by the target.

$\alpha \uparrow$ means lower latency, $\uparrow$ throughput, $\downarrow$ GPU idle time

2. Speculative Token Count (γ): # of tokens draft proposes at each step

3. Acceptance Length (τ): # tokens accepted per round (Avg)

$$\tau = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

- Tips for using SD

- Watch out for memory overhead: On a single GPU, having both draft & target model can squeeze out space for other tasks (like Batch Processing).
- Get the draft model right
  Make sure you fine-tune the draft model on your data to increase Acceptance Rate.
- Don't ignore wasted compute.
  Too many token rejects is bad.

* Prefill - Decode Disaggregation
  ↓ compute ↓ Memory bound

When multiple requests arrive at once, each one has its own prefill & decode stages, but only one can run. When GPU is occupied with compute-heavy prefill tasks, decode has to wait

- Prefill determines TTFT
- Decode determines ITL

So disaggregation:

- Resource re-allocation: Prefill & Decode can run on different hardware. Eg:- If a lot of prompts overlap then KV cache can be reused, so more resources for Decode than on Prefill.
- Parallel Execution & Independent tuning

---

* Prefix Caching (Prompt Caching or Content Caching)

By caching the KV cache of existing query, new query that has the same prefix can skip recomputing that part of prompt.

How it works?

1. During prefill, the model performs a forward pass over entire input & builds KV cache for attention
2. During decode, tokes are generated one-by-one. The attention computes a matrix of token interactions. The resulting KV pairs for each token are stored in GPU.
3. For a new request, with matching prefix you can skip the forward pass for cached part & resume from last part of the prefix.

NOTE:- This only works if the prefix is exactly identical. Even a single character breaks the cache.

- KV cache - For single inference request
- Prefix cache - For multiple inference "

* KV Cache Offloading

It's the process of moving KV cache from GPU memory to lower-cost storage like CPU-memory / SSD / object store. It frees up GPU resources w/o the need for recomputation.

- Why KV cache becomes a bottleneck in LLM inference?

LLMs rely on KV cache to speedup inference. KV cache grows linearly with sequence length. This exhausts GPU memory esp. in long context scenarios.

In fact, not all KV cache is needed in GPU at all times. For eg:- Users might not chat at once, they chat in different hours, so all the KV cache in GPU memory isn't needed during this off-time.

This prevents the model to severe multiple requests from different users. When user resumes interaction the offloaded cache can be loaded into GPU memory.

- Calculation of KV cache size

In transformer-based LLMs, each attn layer stores 2 vectors (a key and a value) for every token in i/p sequence. Each layer has multiple Heads with same dimension.

$$\text{KV cache size (Bytes)} = 2 \times B \times S \times L \times H \times D \times \left(\frac{Q}{8}\right)$$

B: Batch Size; S: Seq. Length; L: # of layers; H: # Heads; D: Head Dim; Q: Quantization

- When to do it?
  - long context
  - Multiple Users / agents
- You can use LMCache.

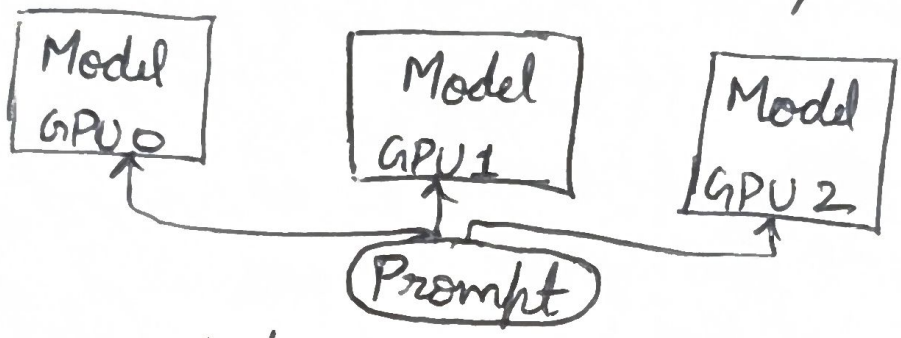
- Metrics
  TTFT

* Parallelism Strategies

It'll allow workloads to be distributed across multiple devices, maximizing throughput and cost efficiency. It's needed as newer models grow in size.

1. Data Parallelism:- (Mostly used in training)

It involves making copies of the model and distributing these replicas across different GPUs. Each GPU only processes part of the input (micro batch) & this process happens in parallel

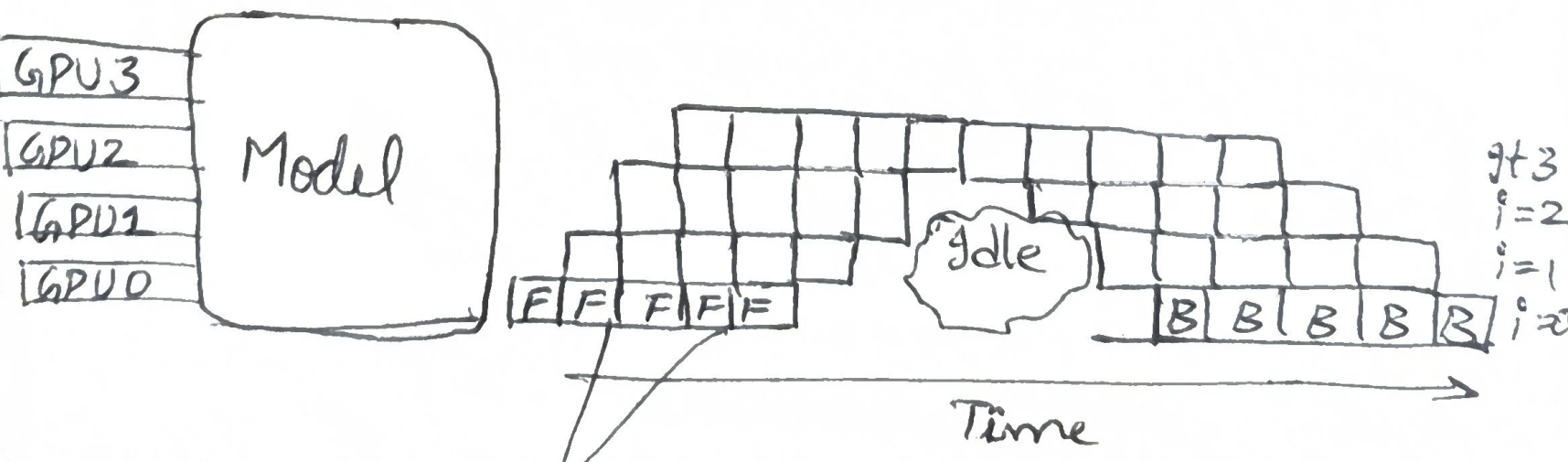


Helps:- lowers latency, ↑ in throughput since multiple requests can be processed at once.

limitations:- Communication overhead
Model size (Large model can't be in single)

2. Pipeline Parallelism:-

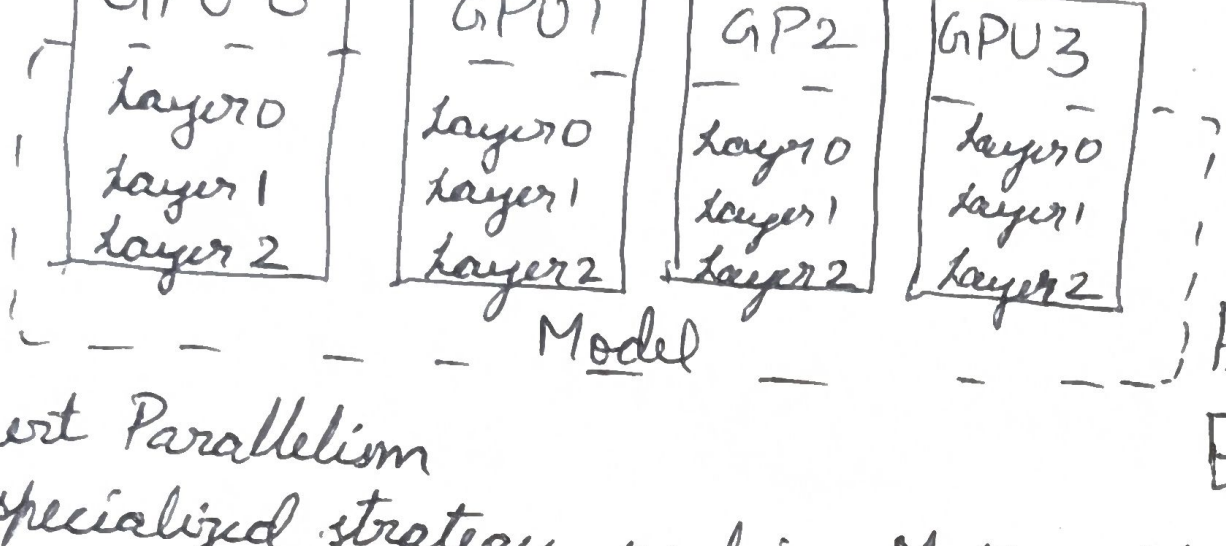
It divides the models layers into sequential chunks, each assigned to a separate device. Data flows through these chunks like an assembly line, with o/p of one going as i/p of another



Micro batching to reduce GPU idle time.

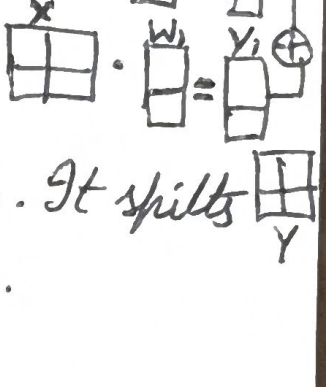
3. Tensor Parallelism

It slices individual layers of the model into smaller blocks. These blocks are processed independently & in parallel.



4. Expert Parallelism

It's specialized strategy used in MoE models. It splits the experts themselves across different devices.



5. Hybrid Parallelism

Eg:- Tensor + Pipeline parallelism.

---

* Offline Batch ~~Processing~~ Inference

It's the process of running models on large, static datasets to generate predictions for batches, rather than one at a time. "Offline", coz it doesn't happen interactively, instead its done as a bulk processing job.

Benefits:-
- Reduces load on real-time systems
- Replaces complex models that are too slow in real-time
- Post-processing & validation of predictions before using in production.

When to use it
- When data doesn't change often.
- Predictions can be stored & retrieved later
- Model too big or slow for real-time